NMAM INSTITUTE OF TECHNOLOGY, NITTE

[Department of Information Science and Engineering]

OPERATING SYSTEMS

PROGRAMMING ASSIGNMENT REPORT

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem

(Semaphore-based and Monitor-based implementations in Java)

SUBMITTED BY

Name	Garthe Anushka Umesh
Registration / USN	NNM24IS084
Course	Operating Systems
GitHub Repository	https://github.com/nnm24is084-collab/OS-Synchronization-Java

SUBMITTED TO

Dr Jason Elroy Martis
Associate Professor
Dept Information Science and Engineering,Nitte

1. Problem Statement and Synchronization Objective

Modern operating systems run many processes and threads at the same time, and these often share data. When access to shared data is not coordinated, the result depends on the unpredictable order in which the scheduler interleaves the threads. This produces **race conditions** (for example a lost update when two threads execute `counter = counter + 1` together), **deadlock** and **starvation** [1][2]. This assignment studies two classical synchronization problems and solves each of them twice, once with **semaphores** and once with a **monitor**, giving four separate Java programs.

1.1 Objective

- Implement the **Readers-Writers** problem with semaphores and with a monitor (`synchronized` methods with `wait()` and `notifyAll()`).
- Implement the **Dining Philosophers** problem for exactly **four philosophers** with semaphores and with a monitor, addressing deadlock prevention and starvation.
- Make every program **generate its own HTML simulation** from the events recorded during its execution, so that the animation is a replay of a real run and not a hand-made drawing.
- Test the programs and explain race conditions, mutual exclusion, deadlock and starvation in relation to each solution.

1.2 The Readers-Writers problem

A shared data object (here an integer counter) is accessed by several concurrent threads. **Readers** only read it; **writers** modify it. Many readers may read together without harm, but a writer needs **exclusive** access. The synchronization requirements are: (i) any number of readers may be inside the critical section at once; (ii) at most one writer may write at a time; (iii) a writer and a reader, or two writers, must never be inside together; and (iv) the policy must be examined for starvation, because favouring one class may postpone the other [1][3].

1.3 The Dining Philosophers problem (four philosophers)

Four philosophers sit at a round table with one fork between each pair of neighbours (four forks). Each alternately thinks and eats, and needs **both** adjacent forks to eat. The forks are the shared, non-shareable resources. If every philosopher picks up the left fork first, all four may hold one fork and wait forever for the other, which is a **deadlock** [4]. A correct solution must prevent or avoid this and must be examined for starvation.

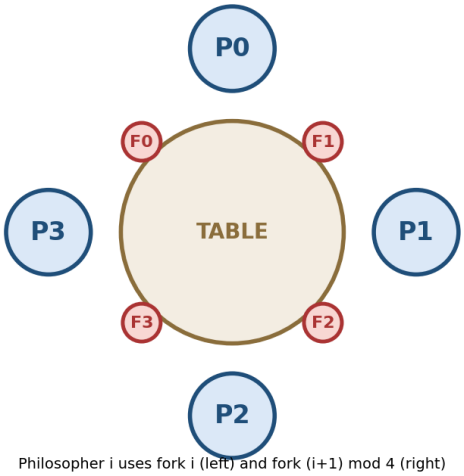


Figure 1: Seating arrangement and fork numbering used in the programs

1.4 Overview of the four implementations

No.	Problem	Technique	Source file
1	Readers-Writers	Semaphores (mutex, resourceLock)	`ReadersWritersSemaphore.java`
2	Readers-Writers	Monitor (synchronized methods + wait/notifyAll)	`ReadersWritersMonitor.java`
3	Dining Philosophers (4)	Semaphores (4 fork semaphores + `room` = 3)	`DiningPhilosophersSemaphore.java`
4	Dining Philosophers (4)	Monitor (to be completed)	`DiningPhilosophersMonitor.java`

Table 1: The four programs

Build and run (Java 15 or newer; the programs use text blocks)

```
javac *.java
java ReadersWritersSemaphore
java ReadersWritersMonitor
java DiningPhilosophersSemaphore
java DiningPhilosophersMonitor
```

Listing 1: Compilation and execution (4 readers, 2 writers, 3 rounds; 4 philosophers, 3 meals; each program writes its HTML file to generated_html/)

Java was chosen because it offers threads, a ready-made counting semaphore (`java.util.concurrent.Semaphore`), and built-in monitors: every Java object has an intrinsic lock, and `synchronized` methods together with `wait()` and `notifyAll()` form the monitor abstraction permitted by the assignment [7]. No external library is needed.

2. Readers-Writers Problem

2.1 Semaphore-Based Implementation

Source: `ReadersWritersSemaphore.java`; output: `generated_html/readers_writers_semaphore.html`.

Shared resource and critical section

The **shared resource** is the integer counter `sharedValue` together with the variable `readCount`, the number of readers currently inside (`activeReaders` and `activeWriters` additionally count the threads inside, for the trace). A reader's **critical section** is the simulated reading time (450-700 ms) followed by reading `sharedValue`; a writer's critical section is the simulated writing time (500-700 ms) followed by `++sharedValue`. The sleeps stand for real work and make the concurrency visible in the simulation.

Table 2: Semaphores of the Readers-Writers semaphore solution

Semaphore	Initial	Purpose
`mutex`	1 (binary)	Protects readCount and activeReaders; only one reader at a time may change them.
`resourceLock`	1 (binary, fair)	Key to the shared resource. A writer holds it alone; the group of readers holds it together (taken by the first reader, returned by the last).

`acquire()` is the wait (P) operation: it decrements the semaphore and blocks if no permit is available. `release()` is the signal (V) operation: it increments the semaphore and wakes one blocked thread [1][7]. The readers act as a **group**: the first reader locks the resource for all readers, and the last reader unlocks it.

```
READER                                  WRITER
------                                  ------
mutex.acquire()                         resourceLock.acquire()
readCount = readCount + 1               ... WRITE (exclusive access) ...
if (readCount == 1)                     resourceLock.release()
    resourceLock.acquire()
mutex.release()
... READ (shared access) ...
mutex.acquire()
readCount = readCount - 1
if (readCount == 0)
    resourceLock.release()
mutex.release()
```

Listing 2: Classical semaphore solution (readers have priority) [3]

```
// READER entry (event recorded while holding mutex, see Section 4)
trace.record(name, "WAITING", "Wants to read (round " + round + ")");
mutex.acquire();  readCount++;
if (readCount == 1) resourceLock.acquire();   // FIRST reader locks for all
activeReaders++;
trace.record(name, "READING", "Entered critical section");
mutex.release();                                // other readers may follow
// ... critical section: Thread.sleep(450..700); read sharedValue ...
// READER exit
mutex.acquire();  activeReaders--;  readCount--;
trace.record(name, "IDLE", "Finished reading");   // recorded BEFORE the release
if (readCount == 0) resourceLock.release();     // LAST reader unlocks
mutex.release();
```

Listing 3: Reader entry, critical and exit sections

Correctness, deadlock and starvation

- **Writer vs. readers:** while any reader is inside, the reader group holds `resourceLock`, so a writer blocks; while a writer is inside, the first arriving reader blocks on `resourceLock`.
- **Writer vs. writer:** only one thread can own the binary semaphore `resourceLock`.

- **Concurrent readers:** after the first reader has taken `resourceLock`, later readers only increment `readCount` and pass.
- **Deadlock:** none. All semaphores are requested in the same order and the only thread that waits while holding `mutex` (the first reader) waits for `resourceLock`, which a writer always releases because it needs no further semaphore.
- **Starvation:** possible, because this program gives priority to readers: if new readers keep arriving, `readCount` never returns to 0, the reader group keeps `resourceLock` and a waiting writer may be delayed indefinitely. No turnstile is used. `resourceLock` is created in fair mode, so threads that already wait for it are served in arrival order, and because every thread performs only a fixed number of rounds the writers are delayed but not postponed forever in this program [3].

2.2 Monitor-Based Implementation

Source: `ReadersWritersMonitor.java`; output: `generated_html/readers_writers_monitor.html`. The monitor is the inner class `SharedMonitor`. All its procedures are `synchronized`, so the intrinsic lock of the monitor object guarantees that only one thread executes a monitor procedure at a time, and `wait()` and `notifyAll()` provide waiting and wake-up on the wait set of the monitor [5][7].

Table 3: State variables and wait/notify mechanism of the Readers-Writers monitor

Monitor element	Kind	Meaning
`activeReaders`	int	Number of readers currently reading.
`activeWriters`	int	1 while a writer is writing, otherwise 0.
`waitingWriters`	int	Writers that want to write; gives writers priority over new readers.
`wait()`	wait set	Readers sleep here while a writer is active or waiting; writers sleep here while any reader or writer is active.
`notifyAll()`	wake-up	Called when the last reader leaves and when a writer leaves; every waiting thread wakes up and re-checks its own condition.

```
synchronized void beginRead(String name) throws InterruptedException {
    while (activeWriters > 0 || waitingWriters > 0)   // writer active OR waiting
        wait();                                       // releases the monitor lock
    activeReaders++;
    trace.record(name, "READING", "Entered read section");
}
synchronized void endRead(String name) {
    activeReaders--;
    trace.record(name, "IDLE", "Finished reading");
    if (activeReaders == 0) notifyAll();              // wake waiting writers
}
synchronized void beginWrite(String name) throws InterruptedException {
    waitingWriters++;                                 // writers have priority
    try { while (activeReaders > 0 || activeWriters > 0) wait();
          activeWriters = 1; }
    finally { waitingWriters--; }
    trace.record(name, "WRITING", "Entered write section");
}
synchronized void endWrite(String name) {
    activeWriters = 0;
    trace.record(name, "IDLE", "Finished writing");
    notifyAll();                                      // wake readers and writers
}
```

Listing 4: Procedures of the Readers-Writers monitor (simplified)

Wait and notify. `wait()` atomically releases the monitor lock and puts the thread into the wait set of the monitor object; on wake-up the thread re-acquires the lock. For this reason `wait()` is always inside a `while` loop and never an `if`: the state may have changed in between, and spurious wake-ups are permitted by the Java specification [7]. This monitor has one wait set shared by readers and writers, so

`notifyAll()` is used: when the **last reader leaves**, and when a **writer leaves**, all waiting threads wake up and each re-checks its own condition. A waiting writer enters when no reader or writer is active; readers enter only when no writer is active **or waiting**, which gives writers priority and lets all woken readers read concurrently.

3. Dining Philosophers Problem

3.1 Semaphore-Based Implementation

Source: `DiningPhilosophersSemaphore.java`; output: `generated_html/dining_philosophers_semaphore.html`. Philosopher *i* uses fork *i* (left) and fork *(i+1) mod 4* (right). The forks are the shared resources; eating is the critical section.

Table 4: Semaphores of the Dining Philosophers semaphore solution

Semaphore	Initial	Purpose
`fork[0..3]`	1 each	One binary semaphore per fork: 1 = on the table, 0 = held by a philosopher.
`room`	3 (N - 1)	Counting semaphore acting as a ticket for the table: at most three philosophers may try to pick up forks at the same time; the fourth blocks on `room`.

Deadlock prevention

A naive left-then-right solution deadlocks when all four philosophers take their left fork simultaneously, a **circular wait** [4][6]. The `room` semaphore limits the competing philosophers to **N - 1 = 3**. The philosopher outside holds no fork (forks are taken only after `room` is acquired), so a chain of waiting philosophers always ends at a free fork or at a philosopher who is eating. A cycle would need all four philosophers, but only three are inside. The eating philosopher finishes, releases his forks and leaves the room, which lets the others continue.

```
room.acquire();                    // at most 3 pass; the 4th BLOCKS here
log(p, "ACQUIRING", "Entered room; taking F" + left + " and F" + right);
forks[left].acquire();             // wait() on left fork
log(p, "ACQUIRING", "Picked up left fork F" + left);
forks[right].acquire();            // wait() on right fork
log(p, "ACQUIRING", "Picked up right fork F" + right);
meals[p] = m;
log(p, "EATING", "Eating meal " + m + " of 3");    // CRITICAL SECTION
Thread.sleep(600);
forks[right].release();            // signal() right fork, then log "put down"
forks[left].release();             // signal() left fork, then log "put down"
room.release();                    // leave the room
```

Listing 5: One eating cycle of a philosopher (semaphore version)

Starvation considerations. All semaphores are created in fair mode (`new Semaphore(n, true)`), so waiting threads are served in FIFO order and cannot be overtaken indefinitely. The number of meals of each philosopher is counted and shown in the simulation, and the program ends only after all four threads have completed their three meals. FIFO queues together with the deadlock-free structure give bounded waiting in practice, although this is not a formal proof for every possible schedule [3].

3.2 Monitor-Based Implementation

TO BE COMPLETED: the source file DiningPhilosophersMonitor.java was not available when this report was prepared. Describe here the source file, the design of the monitor (shared state, lock, wait/notify), a table of its elements, a listing of its main procedures, and the correctness, deadlock and starvation discussion.

What the simulation (Figure 6) already shows: in the event log a philosopher picks up **both** adjacent forks in one step (for example "P2 picked up forks 2 and 3 and started eating"), the shared table is protected by the monitor lock, and the page reports that adjacent philosophers never eat together.

4. Event Recording and HTML Simulation Generation

The assignment requires the visualization to be generated from the execution data of the synchronization program itself. All four programs therefore follow the same pipeline: the threads record events while they run, the program converts the events to JSON, inserts them into an HTML template and writes the file to generated_html/.

4.1 Shared helper classes

- **sharedValue** - the static integer counter of the Readers-Writers programs. Readers read it and each writer increments it once per round (2 writers x 3 rounds = 6). After all threads have finished, the program prints the final value (expected 6) and throws an exception if a reader or writer is still marked as inside the resource.
- **Event recording** - every program records events with a `synchronized` method (the nested classes `Event` and `TraceRecorder` in the Readers-Writers programs, the method `log` with a shared event list in the Dining Philosophers semaphore program). Each event stores a millisecond timestamp, the actor, the state and a message, and is also printed to the console as execution evidence.
- **HTML generation** - `generateHtml()` (Readers-Writers) and `makeHtml()` (Dining Philosophers) hold the fixed HTML/CSS/JavaScript template as a Java text block, replace a placeholder (`__TRACE__` or `EVENTS`) with the JSON array of recorded events, and write the file into `generated_html/`. The path of the file is printed to the console and the page is opened in a browser.

```
{"time":2281,"actor":"R4","state":"READING","detail":"Entered critical section; readers
inside = 1","readers":1,"value":2}
["932","P0","RELEASING","Put down left fork F0","0","1"]   // time, actor, state,
message, philosopher, meals
```

Listing 7: Event formats: one object per event (Readers-Writers) and one array per event (Dining Philosophers)

4.2 The ordering rule

An event is recorded **while the thread still holds** the semaphore or monitor that makes it true. In the Readers-Writers semaphore program a reader records IDLE (finished reading) just *before* it releases `resourceLock`, and a writer records IDLE before `resourceLock.release()`; in the monitor program every `trace.record` call is made inside a `synchronized` method. If an event were recorded after the release, another thread could already have entered and logged its own event first, and the trace would show an impossible overlap. In the Dining Philosophers semaphore program the two "put down" messages are logged immediately after the forks are released, so in principle a neighbour's "picked up" message could appear first; this does not change which philosophers can eat, but the replay could then show a fork with a wrong owner for a moment.

4.3 Replay in the browser

The Readers-Writers pages contain the line `const TRACE = [ ... ]` with the real events of the run, and the Dining Philosophers page contains the corresponding event array. The JavaScript player provides the event log, a timeline with one coloured lane per thread, and the controls Play/Pause, Step, Reset (replay), speed (0.5x to 4x) and a scrub bar; clicking a log line jumps to that event [8]. A problem-specific part draws the scene: reader and writer cards with the shared-resource box and waiting queue (Readers-Writers), or a round table with four philosophers and four forks in SVG together with a philosopher table and a fork table (Dining Philosophers). The displayed state is never stored; it is **rebuilt** from the events up to the current time: for every thread the last event not later than the current time gives its state, and the page checks the safety invariant.

```
function stateAt(actor, t) {                    // last event of this thread up to time
t
  let found = null;
  for (const e of TRACE) if (e.actor === actor && e.time <= t) found = e;
  return found;
}
const reading = states.filter(([a, e]) => e && e.state === "READING");
const writing = states.filter(([a, e]) => e && e.state === "WRITING");
const violation = (reading.length > 0 && writing.length > 0) || writing.length > 1;
```

Listing 8: State of each thread at time t and the safety check (Readers-Writers page)

For the Dining Philosophers semaphore page the events 0..i are replayed: the "picked up" and "put down" messages give the owner of every fork, and the page shows a fork table and a line "Deadlock check" that reports how many philosophers are eating. This line is informational and does not detect a deadlock by itself; deadlock freedom follows from the argument in Section 3.1 and from the fact that the program terminates. Because the whole animation is a function of the recorded events, a run with different timing produces a different animation, so nothing in the HTML tells a pre-written story. Running the program again regenerates the file, which makes the HTML reproducible from program execution.

5. Execution Traces / State Sequences

The excerpts below are taken from the event logs shown in the generated HTML simulations (the same events that each program prints on the console). Each line has the form [time in ms] actor state message. Thread scheduling is non-deterministic, so exact numbers vary between runs, but the properties discussed hold in every run.

5.1 Readers-Writers, semaphore version (4 readers, 2 writers, 3 rounds)

```
[ 1597 ms] W1 WRITING  Entered critical section (exclusive access)
[ 1878 ms] W2 WAITING  Wants to write (round 2)
[ 2280 ms] W1 WRITING  Updated shared value to 2
[ 2280 ms] W1 IDLE     Finished writing; resource released
[ 2281 ms] R4 READING  Entered critical section; readers inside = 1
[ 2281 ms] R2 READING  Entered critical section; readers inside = 2
[ 2282 ms] R3 READING  Entered critical section; readers inside = 3
[ 2282 ms] R1 READING  Entered critical section; readers inside = 4
[ 2767 ms] W1 WAITING  Wants to write (round 2)
[ 2830 ms] R4 READING  Reading shared value 2
[ 2834 ms] R4 IDLE     Finished reading; readers left inside = 3
```

Listing 9: Excerpt of the trace of Program 1 (event log of the simulation)

W1 enters the critical section at 1597 ms and updates the counter to 2 at 2280 ms. W2 asked to write at 1878 ms and has to wait. When W1 releases `resourceLock` at 2280 ms, the readers that had been waiting for it (see the timeline in Figure 3) enter within 2 ms: **readers inside** rises 1, 2, 3, 4, so four readers read the value 2 at the same time while W2 keeps waiting. This is the reader-priority behaviour of the semaphore solution: the reader group holds `resourceLock` until its last member leaves. State sequence: {W1 writing} -> {R4, R2, R3, R1 reading, W2 waiting}. A writer is never inside together with a reader or another writer.

5.2 Readers-Writers, monitor version (4 readers, 2 writers, 3 rounds)

```
[    7 ms] R1 IDLE     Ready to read
[   18 ms] W2 IDLE     Ready to write
[   19 ms] W1 IDLE     Ready to write
[   20 ms] R4 IDLE     Ready to read
[   21 ms] R3 IDLE     Ready to read
[   22 ms] R2 IDLE     Ready to read
[  190 ms] R4 WAITING  Wants to read (round 1)
[  191 ms] R4 READING  Entered read section; readers inside = 1
[  236 ms] R2 WAITING  Wants to read (round 1)
[  237 ms] R2 READING  Entered read section; readers inside = 2
[  261 ms] R3 WAITING  Wants to read (round 1)
```

Listing 10: Excerpt of the trace of Program 2 (start of the run)

All threads are ready within 22 ms. R4 asks to read at 190 ms and enters at 191 ms (readers inside = 1); R2 asks at 236 ms and enters at 237 ms while R4 is still reading (readers inside = 2), so readers read **concurrently** as long as no writer is waiting or active. The writer-priority policy becomes visible in the timeline of Figure 4: from about 1 s to about 4.6 s all four readers stay in WAITING while W1 and W2 take turns writing, and the readers continue only after the last writing segment has ended, because new readers must wait whenever `waitingWriters > 0` or a writer is active.

5.3 Dining Philosophers, semaphore version (4 philosophers, 3 meals)

```
[  931 ms] P1 ACQUIRING  Picked up right fork F2
[  932 ms] P0 RELEASING  Put down left fork F0
[  932 ms] P3 ACQUIRING  Picked up right fork F0
[  933 ms] P2 ACQUIRING  Entered room; trying to take forks F2 and F3
[  934 ms] P0 THINKING   Back to thinking
[  935 ms] P0 THINKING   Thinking
[  936 ms] P1 EATING     Eating meal 1 of 3
[  937 ms] P3 EATING     Eating meal 1 of 3
[ 1193 ms] P0 HUNGRY     Hungry; trying to enter room
```

Listing 11: Excerpt of the trace of Program 3 (event log of the simulation)

P1 picks up its right fork F2 at 931 ms and eats from 936 ms with forks F1 and F2. P0 puts down F0 at 932 ms and P3 picks it up at once as its right fork, so P3 holds F3 and F0 and eats from 937 ms: P1 and P3 are not neighbours, so they eat **at the same time**. P2 enters the room at 933 ms because P0 has left it, and tries to take F2 and F3, which are held by P1 and P3, so it has to wait. P1, P2 and P3 now occupy all three places of the room, so P0, hungry again at 1193 ms, cannot enter (Figure 5), and the circular wait of four philosophers cannot form.

5.4 Dining Philosophers, monitor version (4 philosophers, 3 meals)

```
[    8 ms] P2 is hungry
[    9 ms] P2 picked up forks 2 and 3 and started eating
[   10 ms] P3 is hungry
[   11 ms] P0 finished meal 1 and released both forks
[   12 ms] P0 is thinking
[   13 ms] P2 finished meal 1 and released both forks
[   14 ms] P1 picked up forks 1 and 2 and started eating
[   15 ms] P3 picked up forks 3 and 0 and started eating
[   16 ms] P2 is thinking
```

Listing 12: Excerpt of the trace of Program 4 (event log of the simulation)

P2 is granted forks 2 and 3 together at 9 ms. After P0 and P2 have finished their first meals and released both forks (11 ms and 13 ms), P1 is granted forks 1 and 2 at 14 ms and P3 forks 3 and 0 at 15 ms. P1 and P3 are not neighbours, so they eat **at the same time**, and in this excerpt no philosopher holds a single fork. With four forks, at most two non-adjacent philosophers can eat at the same time. TO BE COMPLETED: add the explanation of the monitor logic after DiningPhilosophersMonitor.java is available.

6. Synchronization Concepts

6.1 Race Condition

A race condition exists wherever two threads access the same data without ordering and at least one of them writes [1][2]. In the Readers-Writers programs the potential races are on the counter `sharedValue` and, in the semaphore version, on `readCount`. `readCount` is protected by `mutex`; the counter is written only by a writer that holds `resourceLock` (semaphore version) or has been admitted by the monitor conditions (monitor version), and the update itself is additionally placed in a short `synchronized` block or `synchronized` method. In the philosophers programs the shared items are the forks (protected by fork semaphores) and the table state of the monitor. The trace recorder is also shared and is protected by `synchronized` methods. Without these protections two writers, or a reader and a writer, could be inside together; the HTML page would then show a red SAFETY VIOLATION and overlapping segments in the timeline.

6.2 Mutual Exclusion

Program	What is mutually exclusive	Mechanism
1 RW semaphore	Writer vs. readers; writer vs. writer; updates of `readCount`	`resourceLock` (group lock for readers) and `mutex`
2 RW monitor	Writer vs. readers; writer vs. writer	Monitor lock (synchronized) plus wait conditions on activeWriters / activeReaders
3 DP semaphore	Each fork is used by one philosopher	`fork[i]` binary semaphores
4 DP monitor	Neighbouring philosophers never eat together, so each fork has one user	`Monitor lock; both forks granted together (see Section 3.2)`

Table 7: Mutual-exclusion mechanisms

The HTML player verifies the property independently: for every moment it takes the state of each thread and shows "SAFETY VIOLATION" and "OVERLAP DETECTED" if a writer overlaps a reader or another writer. This message never appeared for the correct programs; the pages in Figures 3 and 4 show "no reader/writer or writer/writer overlap".

6.3 Deadlock

A deadlock requires all four Coffman conditions: **mutual exclusion**, **hold and wait**, **no preemption** and **circular wait** [6]. Preventing any one of them is sufficient. Both Readers-Writers solutions are deadlock-free because the lock order is fixed and a lock is held only while waiting for one that is always released. In the semaphore philosophers solution, mutual exclusion, hold-and-wait and no-preemption exist (a philosopher holds one fork while waiting for the other), but **circular wait** is broken by the `room` semaphore (at most N - 1 = 3 competitors). In the monitor solution **hold-and-wait** is avoided because both forks are granted together (see Section 3.2). All programs terminated normally in the runs made for this report, which would be impossible after a deadlock.

6.4 Starvation

Program	Policy	Who may starve	Remedy
1 RW semaphore	Readers priority	Writers, if readers keep arriving	`Add a turnstile semaphore that blocks new readers while a writer waits (not implemented here)`
2 RW monitor	Writers priority	Readers, if writers keep arriving	Alternate turns or limit consecutive writers

Program	Policy	Who may starve	Remedy
3 DP semaphore	Fair FIFO semaphores, `room` = 3	Practically nobody; FIFO gives bounded overtaking	Already fair; meals are counted
4 DP monitor	To be completed (Section 3.2)	To be completed (Section 3.2)	To be completed (Section 3.2)

Table 8: Starvation analysis

Deadlock means that nobody can proceed; starvation means that the system keeps progressing while one thread is postponed indefinitely. Bounded waiting, the third requirement of the critical-section problem, is the property that rules starvation out [1].

7. Comparison of Semaphore and Monitor Approaches

Both families were implemented for the same two problems, which allows a comparison based on actual code instead of theory alone [1][2][5].

Aspect	Semaphore approach	Monitor approach
Abstraction level	Low level: an integer with wait and signal	High level: shared data, procedures, lock and condition variables in one unit
Mutual exclusion	Programmer must pair `acquire` and `release` correctly everywhere	Automatic: only one thread is inside the monitor at a time
Waiting for a condition	Encoded indirectly through counters and extra semaphores (readCount, room)	Explicit and readable: `while (condition) await()`
Typical errors	Forgotten `release`, wrong order (deadlock), unbalanced counts	`if` instead of `while`; missing signal causes a lost wake-up
Readers-Writers code	Two semaphores plus a counter; "first reader locks, last unlocks" is subtle	Three state variables and one wait set with notifyAll(); rules read directly as conditions
Dining Philosophers code	Four fork semaphores plus `room`; one fork acquired after the other	Table state under the monitor lock; both forks granted together (see Section 3.2)
Deadlock handling	Limit entry to N - 1 (circular wait removed)	Atomic acquisition (hold-and-wait removed)
Policy control	Hidden in the order of semaphore operations	Visible in the conditions and in which condition is signalled
Performance	Very light weight; good for simple resource counting	Slightly more overhead from lock and condition handling; negligible here

Table 9: Comparison of semaphores and monitors

Summary. Semaphores are the more primitive tool and can express every synchronization pattern, but correctness depends on the careful placement of each operation and the intention is not written in the code. Monitors keep the shared state and the access rules together, which makes monitor programs easier to read, reason about and modify; for example, switching the Readers-Writers monitor from writer priority to reader priority only requires changing one condition. Their drawbacks are the dependence on language or library support and the discipline of the `while` loop. In Java, `synchronized` methods with `wait()` and `notifyAll()` supplied the monitor abstraction without difficulty [7]; the price of the single wait set is that `notifyAll()` wakes every waiting thread and each of them must re-check its condition.

8. Results and Observations

8.1 Readers-Writers

Program	Events	Run time	Result
1. RW semaphore (4 readers, 2 writers, 3 rounds)	84	about 5.9 s	No overlap detected in the simulation; the program prints the final value (expected 6)
2. RW monitor (4 readers, 2 writers, 3 rounds)	84	about 6.1 s	No overlap detected in the simulation; the program prints the final value (expected 6)

Table 10: Results of the default Readers-Writers executions

- **Mutual exclusion held:** in the runs shown in Figures 3 and 4, the writers' WRITING segments in the timeline never overlap READING segments or each other, and the page reports no overlap.
- **Reader concurrency observed:** up to four readers were inside the critical section at the same time in the semaphore version (readers inside = 4 at 2282 ms), and in the monitor version R4 and R2 were inside together at the start of the run (readers inside = 2).
- **Policy difference visible:** the semaphore version let readers keep the resource until the last one left (reader priority); the monitor version made new readers wait behind waiting writers (writer priority), so all four readers waited while both writers completed their rounds (Figure 4).

8.2 Dining Philosophers

Program	Events	Run time	Result
3. DP semaphore (4 philosophers, 3 meals)	112	about 4.6 s	Terminated; every philosopher completed 3 meals; P1 and P3 ate together (Figure 5)
4. DP monitor (4 philosophers, 3 meals)	49	fill in	Confirm from the console output: all meals completed, no deadlock

Table 11: Results of the default Dining Philosophers executions

- **No deadlock:** the semaphore program terminated after all four philosophers finished their three meals, which would be impossible after a deadlock. Confirm the same for the monitor program from its console output.
- **No starvation observed:** the semaphore program ends only after all four philosophers have completed their requested number of meals.
- **Different eating patterns:** the semaphore version showed philosophers holding a single fork while waiting for the second (a window that the `room` limit makes safe); the monitor version never showed a philosopher holding only one fork.
- **Concurrency in eating:** at most two non-adjacent philosophers (P0 and P2, or P1 and P3) ate at the same time.

Testing methods and limitations

The programs were tested by (i) functional runs of all four programs, (ii) the final-value check printed at the end of the Readers-Writers programs, (iii) the termination check (all threads must finish) and the meal counters in the philosophers simulation, (iv) the safety checks inside the HTML pages, and (v) opening the generated files in a browser to confirm that the simulations run. Known limitations: thread scheduling is non-deterministic, so event order and times differ from run to run; the HTML is a **replay** generated after the run, not a live view; the "Deadlock check" line of the philosophers page is informational; and the default policies are not starvation-free for every possible schedule.

9. HTML Simulation Screenshots

Each program writes its own HTML file into `generated_html/`. The pages show the state of every thread, the shared resource or fork ownership, an event log and a timeline with colour coding that is consistent across cards, tables and lanes. Controls: Play/Pause, Step, Reset, speed selector and scrub bar.

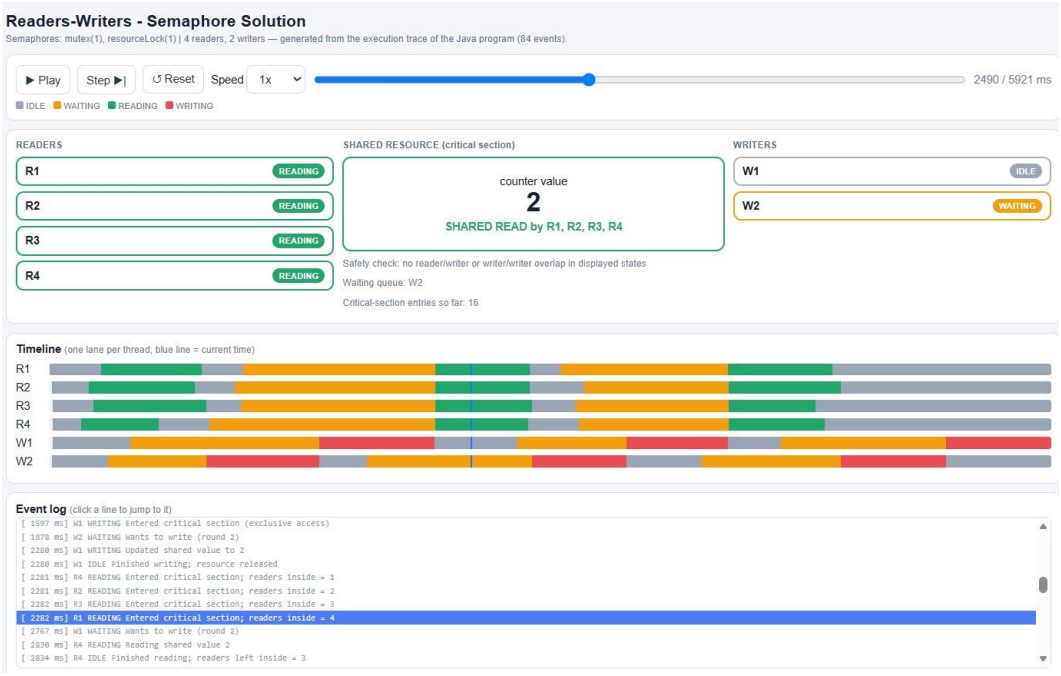


Figure 3: Program 1 - Readers-Writers (semaphores). All four readers (R1-R4) read concurrently while writer W2 waits and W1 is idle; counter = 2 ("SHARED READ by R1, R2, R3, R4").

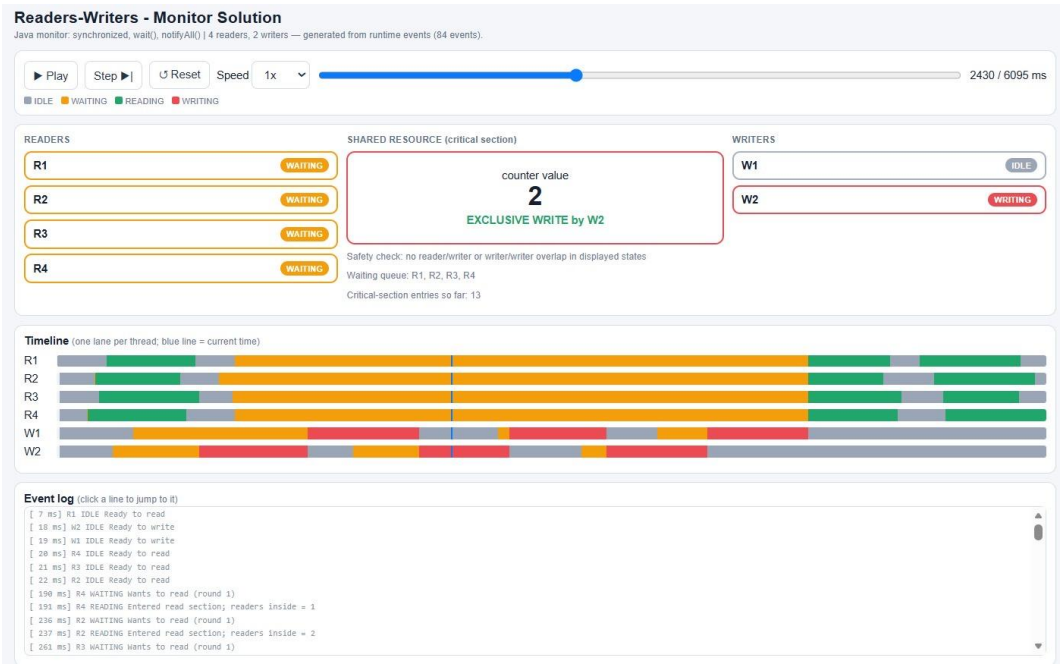


Figure 4: Program 2 - Readers-Writers (monitor). W2 holds the resource exclusively ("EXCLUSIVE WRITE by W2") while readers R1-R4 wait and W1 is idle; counter = 2 and the safety check shows no overlap.

In Figures 3 and 4 the writers' red WRITING segments never overlap the readers' green READING segments, and the two writers never overlap each other, which is visual evidence of mutual exclusion.

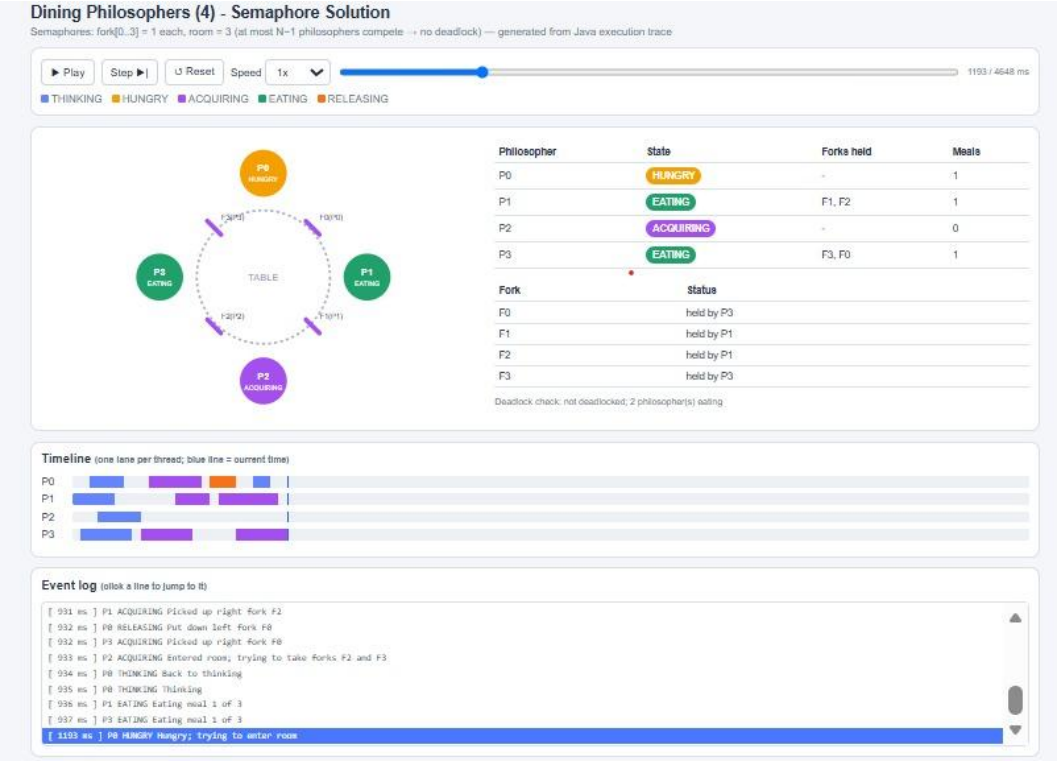


Figure 5: Program 3 - Dining Philosophers (semaphores). P1 and P3 eat at the same time while P2 is ACQUIRING and P0 is HUNGRY (1193 ms of 4648 ms).

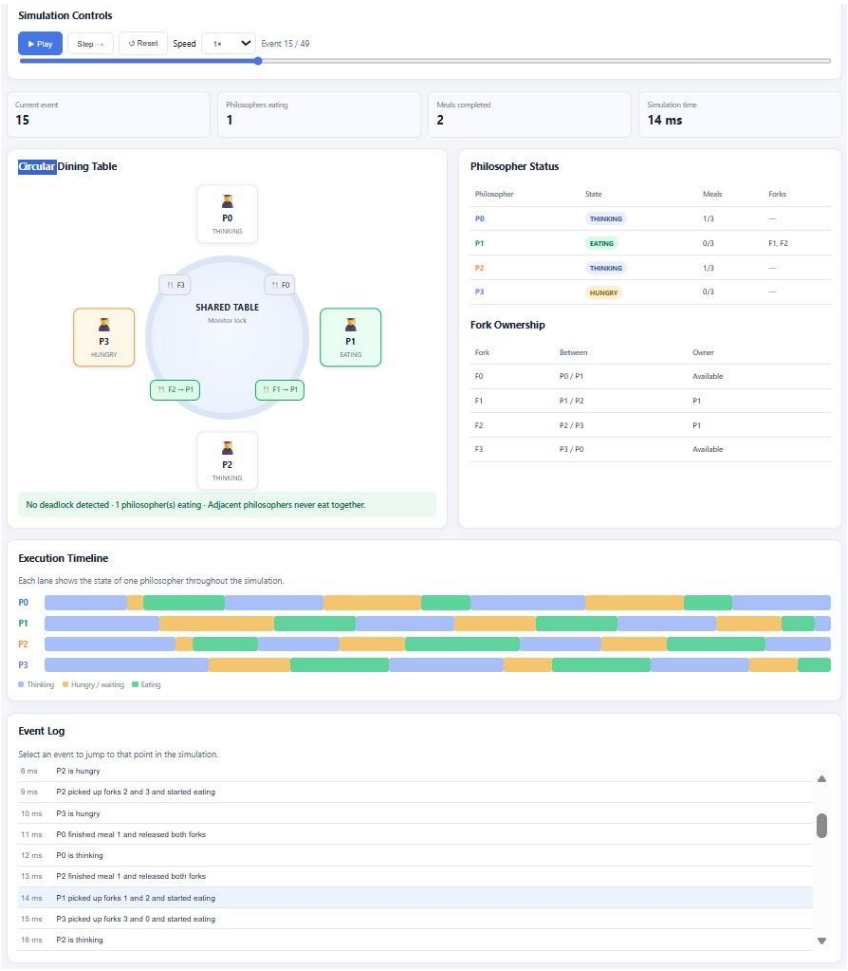


Figure 6: Program 4 - Dining Philosophers (monitor). P1 eats with forks F1 and F2 while P3 is hungry and P0 and P2 are thinking (event 15 of 49).

In Figure 5, P1 and P3 are EATING with forks F1, F2 and F3, F0, P2 is ACQUIRING without a fork and P0 is HUNGRY, waiting to enter the room; all four forks are held, but the system is not deadlocked because two philosophers are eating and will release their forks. In Figure 6, P1 is EATING and holds F1 and F2, and the fork ownership table and the status line ("No deadlock detected") agree. In the timeline of Figure 6 no two neighbouring philosophers have overlapping EATING segments.

10. AI Usage Summary

The assignment permits and encourages AI assistance, provided the student tests, understands and documents the result [9]. This section summarizes the use of AI.

Table 12: AI usage summary

Item	Description
AI tool used	Claude (Anthropic) was used to prepare this report in Word format from the student's Java programs and simulation screenshots. Add any other AI help used for the code.
Initial prompt	Describe your initial prompt here.
Follow-up prompts	Requests to adapt a sample report to the student's own name, USN, screenshots and Java programs.
Manual work by the student	Describe what you did yourself: writing and running the Java programs, taking the screenshots, creating the GitHub repository, and so on.
Verification	Execution of the programs and inspection of the generated HTML simulations (Figures 3-6). Add other checks you made.

Reflection on limitations and improvements

The following limitations and possible improvements were identified in the programs:

- **Order of recording and releasing.** In the Readers-Writers programs events are recorded before the resource is released. In the Dining Philosophers semaphore program the "put down" messages are logged after the release, so the log could show a neighbour's "picked up" message first; logging before the release (Section 4.2) would remove this.
- **Reader priority.** The semaphore Readers-Writers program has no turnstile, so a steady stream of readers could delay a writer; adding a turnstile semaphore would remove writer starvation.
- **Fixed delays.** The semaphore philosophers program uses fixed delays (250 ms thinking, 600 ms eating), so runs look alike; random delays would show more different interleavings.
- **Informational deadlock line.** The "Deadlock check" line of the philosophers page only reports how many philosophers are eating; a real detector (all forks held and nobody eating) could be added.

11. References

[1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, Wiley - chapters on process synchronization, classical synchronization problems and monitors.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, Pearson - section on classical IPC problems (Dining Philosophers, Readers-Writers).

[3] A. B. Downey, *The Little Book of Semaphores*, Green Tea Press - Readers-Writers (turnstile solution) and Dining Philosophers.

[4] E. W. Dijkstra, "Hierarchical ordering of sequential processes", *Acta Informatica*, 1971 - origin of the Dining Philosophers problem.

[5] C. A. R. Hoare, "Monitors: an operating system structuring concept", *Communications of the ACM*, 17(10), 1974.

[6] E. G. Coffman, M. Elphick and A. Shoshani, "System deadlocks", *ACM Computing Surveys*, 3(2), 1971 - the four deadlock conditions.

[7] Oracle, *Java SE 21 API documentation*: `java.util.concurrent.Semaphore`, `java.lang.Object (wait, notifyAll)` and `synchronized`. https://docs.oracle.com/en/java/javase/21/docs/api/

[8] Mozilla Developer Network (MDN) Web Docs - JavaScript, SVG and `requestAnimationFrame`. https://developer.mozilla.org/

[9] Claude (Anthropic) - AI assistant used for documentation drafting.

12. Academic Integrity Declaration

The following declaration is included as required by the assignment:

> "I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own."

The AI tool used in this assignment (Claude, developed by Anthropic) is disclosed in Section 10. All books, documentation and websites consulted are listed in Section 11.

Signature: _____ Name: Garthe Anushka Umesh

Date: _____ USN: NNM24IS084